



# PROJECT-BASED LEARNING

## PHASE-I EVALUATION

### VERI NEWS

Team ID: DSCPP-III-2026-T224

**Department of Computer Science & Engineering**  
Graphic Era (Deemed to be University), Dehradun

Academic Session 2026-27

# Team & Mentor Details

---



## Team Information

**Team ID:** DSCPP-III-T224

**Semester:** 3rd

**Section:** D, J, ML2

**Domain:** Data Structures + OOP / C++

## Team Members

1. Shambhavi Srivastava – 2510012471
2. Abhinav Panwar – 2512510050
3. Akshat Sharma – 2510010700

## Mentor

**Mentor Name:** Mr. Kamal Kumar Gola

**Interactions completed:** \_\_\_\_\_

# Problem Statement & Motivation

---

## Problem Statement

Online information spreads rapidly, making it difficult for users to distinguish reliable news from misleading or suspicious claims. Manual verification is slow and inconsistent.

## Motivation

- ▶ Rapid spread of unverified information
- ▶ Difficulty of checking every claim manually
- ▶ Need for a simple preliminary verification tool
- ▶ Need to compare claims with stored evidence

## Target Users / Stakeholders

- ▶ General internet users
- ▶ Students and researchers
- ▶ News readers
- ▶ Content moderators

## Project Objectives

1. Develop a solution for preliminary news verification.
2. Store and manage verified and suspicious news records.
3. Use DSA for efficient searching, indexing and matching.
4. Apply C++ OOP for modular application design.
5. Generate an explainable credibility / matching score.
6. Evaluate the system with predefined test cases.

**Key Objective:** Provide users with a quick, explainable preliminary assessment of a news claim based on stored records and matching evidence.

# Proposed Solution

---

## System Idea

The system checks a news claim by extracting keywords, finding related stored records, and giving a preliminary result.

## Main Features

- ▶ News verification    • Keyword search
- ▶ Similar-news detection    • Source checking
- ▶ Verification report

## Input

- ▶ Headline or news text
- ▶ Existing news records

## Processing

1. Extract keywords → Hash for lookup
2. Find related news → Search & rank
3. Calculate matching score

## Result

- ▶ Likely Verified    • Needs Verification    • Suspicious

**Stored:** News records | Sources | Status | Search history

# Integration of PBL Course Concepts

## PBL Integration

DSA provides efficient storage and searching, while C++ OOP organizes the system into reusable classes and modules.

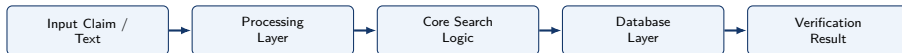
| Concept       | Project Use                                                |
|---------------|------------------------------------------------------------|
| DSA in C      | Linked List, Hashing, BST, Searching, Sorting              |
| OOP with C++  | Classes, Objects, Encapsulation, Inheritance, Polymorphism |
| File Handling | News records, user activities, and verification logs       |
| Database      | News data and verified source records                      |

## DSA Module

- ▶ **Linked List:** News records
- ▶ **Hash Table:** Fast keyword lookup
- ▶ **BST / Queue:** News indexing & requests
- ▶ **Sorting:** Match scoring and ranking

## C++ Classes

- ▶ **User & News**
- ▶ **NewsDatabase & NewsAnalyzer**
- ▶ **VerificationEngine**
- ▶ **Report**



## Data / Control Flow

- ▶ User inputs claim or article text for verification.
- ▶ Text is cleaned and keywords are extracted.
- ▶ Hashing and BST search find related stored records.
- ▶ Relevance scoring generates a final classification.

## Major System Components

- ▶ **Input Module:** Receives raw headline or claim.
- ▶ **Analyzer Module:** Tokenization and stopword removal.
- ▶ **Matching Engine:** Compares keywords with database.

## Processing and Storage Pipeline

- ▶ **Input Layer:** Claim text entered by the user.
- ▶ **Processing Layer:** Validates structure and extracts tokens.
- ▶ **Core Logic:** Hash lookup, searching, and credibility score.
- ▶ **Database Layer:** Stores known news and trusted sources.
- ▶ **Output Layer:** Credibility score and summary status.

# Technology Stack & Methodology

---

## Technology Stack

- ▶ **C & C++:** DSA in C and OOP structuring in C++.
- ▶ **DSA:** Hashing, BST, searching, and sorting.
- ▶ **File Handling:** Stores local database and logs.
- ▶ **VS Code / Code::Blocks:** IDE and build environment.
- ▶ **Git & GitHub:** Version control and team tracking.

## Purpose

Build a modular, searchable, and explainable news verification prototype.

## Development Methodology

1. **Requirements:** Define parameters, inputs, and outputs.
2. **Design:** Model classes, hash functions, and trees.
3. **Implementation:** Build modular C/C++ components.
4. **Testing:** Verify keyword search with edge cases.
5. **Evaluation:** Measure score accuracy and latency.

**Pipeline:** Requirements → Design → Implementation → Testing → Evaluation

# Initial Progress & Team Contribution

---

## Progress Achieved

- ▶ Problem statement identified
- ▶ Background study completed
- ▶ System requirements drafted
- ▶ Architecture and data flow designed
- ▶ Core DSA concepts mapped
- ▶ Database file schemas established
- ▶ Prototype implementation scheduled

## Individual Contribution

| Member               | Role           | Contribution                                                   |
|----------------------|----------------|----------------------------------------------------------------|
| Shambhavi Srivastava | Team Lead      | Project planning, requirements, and system design coordination |
| Abhinav Panwar       | Developer      | C++ OOP classes, indexing, and news data management            |
| Akshat Sharma        | Testing / Docs | Test suite design, verification metrics, and documentation     |

# Roadmap, Expected Outcomes & References

---

## Roadmap

1. **Phase-I (Analysis & Design):** Define news schemas, select DSA/OOP structures, and design verification pipeline.
2. **Phase-II (Development):** Implement modules, hash indexing, matching engine, and file-based data store.
3. **Phase-III (Testing & Finalization):** Run structured news test cases, optimize search times, and deliver working tool.

## Expected Outcomes

- ▶ Working TruthCheck preliminary verification engine
- ▶ Sub-second keyword lookups via hashing
- ▶ Transparent, rule-based credibility indicators
- ▶ Foundation for future automated NLP ingestion

## References

1. Data Structures & Algorithms – Course Notes
2. Object-Oriented Programming with C++ – Reference Guide
3. ISO/IEC C++ Standard Documentation
4. GitHub Repository for Team Version Tracking

**Thank You**

Questions & Suggestions